

ChronoSquares Rebuild — Replit Agent Build Prompt

Paste this entire document into the Replit Agent chat as your instructions. Follow it in order. Do not skip the "Non-Negotiables" section — read it first, every time, before touching any code.

0. Context (read this first)

This is a **repositioning**, not a from-scratch app. The existing ChronoSquares codebase was built around personal productivity. That direction is being killed. The new ChronoSquares is a **shared client-delivery platform for service-based agencies** (5–50 active clients: marketing, creative, consulting, dev, and operational agencies running retainer or project work).

The core problem it solves: agencies run client delivery across email, spreadsheets, forms, and disconnected tools. ChronoSquares gives the agency, the client, and any contractors/partners **one shared workspace** with role-based views, so nobody is

chasing status updates, files, approvals, or invoices across six different tools.

You are reusing the visual identity, not the product logic. The landing page design, styling system, and admin panel are correct and must not be redesigned. Everything else — data model, pages, copy, navigation, core objects — is being rebuilt around the new purpose.

1. Non-Negotiables — DO NOT VIOLATE THESE

1. **Preserve the existing landing page's visual design exactly:** layout structure, spacing, typography, color palette, component styles, animations/transitions, and overall aesthetic. Do not introduce a new design system, new fonts, new color scheme, or new layout grid.
 - What changes on the landing page is **copy/content only:** headlines, subheadlines, feature descriptions, testimonials/logos placeholders, CTA text — all rewritten to describe the agency client-delivery platform, not personal productivity.
 - Before editing the landing page, inspect the current component structure and reuse the

same components/sections. Do not delete and rebuild sections from scratch — edit content inside the existing structure.

2. **Preserve the existing admin panel exactly as is.**

Same layout, same navigation, same styling, same functionality it currently has. Do not redesign, simplify, or restructure it. If new admin capabilities are needed later for the new product (e.g. managing provider organisations), they get **added** as new sections/pages inside the existing admin panel pattern — never as a redesign of what's there.

3. **Do not silently change the design system**

(spacing scale, border radius, shadows, color tokens) anywhere in the app while building new screens. New screens must visually match the existing landing page and admin panel — same design tokens, same component library, same look and feel throughout the whole product, not just the marketing page.

4. Before writing any code, **list out loud** (in the Replit Agent's own response) what you found in the existing codebase: which framework, which styling approach (Tailwind config, CSS variables, component library), which folders hold the landing page vs admin panel vs app pages. Confirm this matches what's described here before

proceeding. If something doesn't match, stop and ask rather than guessing.

5. Work in small, reviewable increments. After each major phase (see Section 8), stop and summarize what was built before moving to the next phase. Do not attempt to build the entire product in one uninterrupted pass.

2. Product Summary

ChronoSquares is a shared client-delivery platform for managing the full relationship between a service provider (agency), their client, and any other involved parties (contractors, partners, external stakeholders).

Core value:

- One reliable source of truth for everyone involved in a client relationship
- Visibility into responsibilities, progress, costs, and decisions
- Automated handovers and admin (reminders, status changes, notifications)
- A professional, reassuring client-facing experience
- The agency can manage full operational detail without overwhelming the client's view

Target customer (initial niche):

- Service-based agencies with roughly 5–50 active clients
 - Retainer-based or project-based work
 - Currently running onboarding/delivery manually across email, spreadsheets, forms, docs, and disconnected task tools
-

3. Roles and Access (this drives all permissions logic)

Every person gets a **role-based view** — only what's relevant to them, nothing else.

Role	Sees / does
Service Provider (agency)	Full oversight: all their clients, projects, finances, risks, tasks, internal operations. This is the "admin" side of the product for the agency (not the platform admin panel — a separate concept, see Section 7).
Client	Their own project(s) only: progress, requests to complete, approvals, files, agreed costs. No visibility into agency-internal data, other clients,

Role	Sees / does
	or internal task detail not meant for them.
Contractor / Partner	Only the work assigned to them: deadlines, relevant files, relevant communication. No visibility into finances or unrelated project areas.
Other Stakeholder	Invited ad hoc for a specific approval, deliverable, or stage. Minimal, scoped access — expires or is scoped to that specific item.

Build this as a proper permission layer from day one
— not something bolted on later. Every object (project, task, file, message, invoice) needs an owner-role and a visibility rule. Get this data model right before building UI on top of it.

One provider can manage multiple client organisations. One client can have access to multiple active/completed services. A project can involve internal team + client + contractors + external partners simultaneously, each seeing only their scoped slice.

4. The 7 Squares (core product structure)

These are the seven functional areas that make up the full service lifecycle. Structure the app's information architecture around these seven areas – they should be recognizable as the main navigation for the Provider view.

1. **Lead Square** – enquiries, qualification, discovery, proposals, quotations
2. **Onboarding Square** – intake forms, contracts, initial payment, document collection, kickoff
3. **Project Square** – milestones, tasks, responsibilities, dependencies, progress
4. **Communication Square** – updates, questions, decisions, approvals, meeting records
5. **Finance Square** – costs, budgets, invoices, payments, financial visibility
6. **Files Square** – contracts, assets, working documents, versions, final deliverables
7. **Care Square** – support, feedback, issue resolution, renewals, ongoing client success

For the MVP (Section 6), you are not building all seven at full depth. Build one complete workflow end-to-end first: Onboarding → Project → Communication

→ Finance (basic) → Files. Lead Square and Care Square come after the core loop works.

5. Client Lifecycle (the flow the product must support)

1. Lead submits an enquiry, or provider adds one manually
2. Provider qualifies the lead, sends a proposal/quotation
3. Client accepts, signs contract, pays, completes onboarding requirements
4. ChronoSquares auto-creates the shared workspace, project structure, tasks, milestones, and requests
5. All parties track delivery, communicate, approve work, exchange files — in one place
6. Project closes: final delivery, payment, feedback, organised record
7. Relationship moves to support/renewal/new project without restarting the whole process

Build the data model so a "workspace" persists across multiple projects for the same client relationship — closing a project should not destroy history or force re-onboarding for a repeat engagement.

6. MVP Scope — Build This First, Nothing More

Do not build the full Workflow Layer or AI Architect in the MVP. Build:

- Provider dashboard (overview of active clients/projects, using existing admin-panel-adjacent styling patterns, not the platform admin panel itself)
- Client portal (client-facing view, scoped per role rules in Section 3)
- Intake form + document collection
- Proposal → contract → initial payment status tracking (status tracking only for MVP — full e-signature/payment provider integration can be stubbed/mockbed initially unless you confirm a specific provider to integrate)
- Project milestones + shared progress view
- Tasks, approvals, files, and status updates
- Basic cost and invoice tracking (line items + status, not a full accounting system)
- Automated reminders and handovers (basic triggers: e.g. "onboarding form submitted → notify provider", "milestone marked done → notify client for approval")

Explicitly **out of scope for MVP**: Lead Square (qualification/quotation flows), Care Square (renewals/support ticketing), the full drag-and-drop Workflow Layer builder, and the AI Architect (natural-language workflow generation). These are documented in Sections 4 and 9 for context, not for building now.

7. Two Separate "Admin" Concepts – Do Not Conflate

Be careful: there are two different things called "admin" here.

1. **Platform admin panel** – the existing admin panel in the current codebase, used to administer the ChronoSquares platform itself (e.g. managing accounts/organisations at the platform level). **Preserve this exactly per Section 1.**
2. **Provider dashboard** – the new agency-facing view where a service provider manages their own clients/projects. This is a normal in-app role view, not the platform admin panel. Build it as a new set of app pages, styled consistently with the existing design system, but it is functionally distinct from the platform admin panel and must not be merged into it.

If you're unsure which one a feature belongs to, stop and ask rather than guessing.

8. Build Order (do not reorder)

1. **Audit phase** — inspect existing codebase, confirm design system, confirm what the landing page and admin panel currently look like and how they're structured. Report back before writing code.
2. **Data model** — set up the core entities: Organisation (provider), Client, Workspace, Project, Task, Milestone, File, Message, Invoice, User, Role, Permission. Get roles/permissions right (Section 3) before building UI.
3. **Landing page copy rebuild** — update copy only, preserve design, per Section 1.
4. **Auth + role-based routing** — provider vs client vs contractor views, each seeing only their scoped data.
5. **Onboarding Square** — intake form, document collection, contract/payment status tracking.
6. **Project Square** — milestones, tasks, progress view (both provider and client-facing versions).
7. **Communication Square (basic)** — updates, approvals, comments/decisions log.

8. **Finance Square (basic)** – cost/invoice line items and status only.
 9. **Files Square** – upload, versioning basics, client-visible vs internal-only files.
 10. **Basic automation/reminders** – the handful of triggers listed in Section 6, nothing more.
 11. Stop. Report the completed MVP. Do not proceed to Lead Square, Care Square, full Workflow Layer, or AI Architect without explicit instruction.
-

9. Reference Only – Not for This Build

The long-term vision includes a **Workflow Layer** (trigger contracts/payments/forms/tasks/reminders, gate stage progression on required conditions, connect external tools) and an **AI Architect** (build workflows from plain-language descriptions, generate onboarding checklists, summarise progress/risk, help locate documents). These are documented here so you understand where the data model needs to leave room to grow, but **do not build these now**. When designing the data model in Section 8, avoid decisions that would make adding these later painful (e.g. keep task/trigger logic reasonably decoupled), but do not build the features themselves.

10. Acceptance Checklist (confirm each before calling MVP done)

- ☐ Landing page: same exact visual design as before, all copy now describes the agency client-delivery platform
 - ☐ Platform admin panel: untouched, same layout/functionality as before
 - ☐ Provider dashboard: new pages, matches existing design system (not a new look)
 - ☐ Client portal: shows only client-scoped data, matches existing design system
 - ☐ Roles/permissions: a client genuinely cannot see another client's data, internal-only tasks, or agency finances not shared with them
 - ☐ Onboarding → Project → Communication → Finance → Files loop works end-to-end for one test client
 - ☐ Reminders/notifications fire for the specific triggers listed in Section 6
 - ☐ No Lead Square, Care Square, Workflow Layer builder, or AI Architect features present yet
-

11. If Anything Is Ambiguous

Stop and ask rather than guessing, especially about:

- Which payment/e-signature provider (if any) to integrate for the MVP vs. mock
- Exact copy/positioning for the landing page beyond the direction given here
- Any place where preserving the existing design conflicts with a new page's requirements